A09｜怎样确认运行条件没有找错?

A01 确认脚本目标；A09 再把**正在执行的文件、工作目录、解释器、依赖与配置**对齐。输出看起来不对时，先确认它属于哪次运行条件，再怀疑业务代码。

改了没生效、装了找不到，先比较什么?

当前问题	关系或短用法	记到什么程度
这次究竟运行谁、从哪里运行、用哪个 Python?	`Path(__file__).resolve()` 报告当前脚本位置；`Path.cwd()` 报告工作目录；`sys.executable` 报告实际解释器。	**三份证据不必相同。** 能对照编辑器文件与实际运行；完整探针查用。
输入 python，就一定是想用的解释器吗?	PowerShell 先解析命令名，可能遇到别名、函数或外部程序。`Get-Command` 查命令入口，运行后的 `sys.executable` 查实际解释器。	名字与实际运行分开核对；不凭“最新安装”或窗口标题猜。
为什么安装成功仍无法导入?	A 的解释器安装包，不等于 B 也能找到。先核对安装和运行是否使用同一入口，再看实际导入文件位置。	**包归属于环境，不是整台电脑共享一个安装结果。** 不同时重装所有环境。
怎样把安装、运行明确绑到同一环境?	让同一条 `python.exe` 路径执行 `-m pip` 和脚本；`-m` 是让这个 Python 运行指定模块。`&` 是 PowerShell 调用后面程序的写法。	逐渐用熟“先选解释器，再让它做事”；无需靠提示符里的环境名猜。
编辑器选了 A，终端会自动一致吗?	编辑器运行、调试与已有终端可能各有设置。分别运行探针，比较解释器证据。	界面选项可查；没红线不证明另一个终端也能导入。

教材探针的三行输出分别读作：`SCRIPT` = 文件，`CWD` = 工作目录，`PYTHON` = 解释器。它们是标签，不是三条需要执行的命令。脚本目录与工作目录不同，还会影响 A08 的相对文件路径。

回看：「这段输出到底属于谁?」「命令中的 python，与真正启动的解释器是一回事吗?」「包装在 A 环境，为什么 B 还是找不到?」「编辑器选了 A，终端会自动跟着变吗?」。

环境一致了，为什么结果仍可能不同?

当前问题	关系或短用法	记到什么程度
环境变量、配置文件、默认值，谁优先?	优先级由代码决定。本例先选环境变量，没有才读文件，文件不存在才用默认值；已提供的非法值拒绝，不悄悄回退。	**既看最终值，也看来源。** 不是所有应用都遵守这个顺序；编辑器设置也不自动成为程序配置。
想找代码，应该搜什么?	找文件名用 `Get-ChildItem`；找文件里的文字用 `Select-String`。前者定位文件，后者定位行。	记选择理由，筛选、递归和管道参数查用；命中不证明该行执行过。
有类型提示、格式整齐，就正确了吗?	`values: list[int]`、`-> int` 表达类型意图，不自动执行运行时校验；formatter 整理排版，静态提示和实际测试另有作用。	分清各工具能证明什么，不为消除提示盲目改环境或放宽业务规则。
怎样让下次找得到同样条件?	留下目录、解释器、依赖安装入口、运行入口和预期观察；不只写“在某终端能跑”。	用项目事实写简短说明；完整版本与重建在 A12 接上。

回看：「配置都叫 mode，谁的值会生效？」「文件越来越多，怎样找到真正要看的位置？」「格式整齐、没有红线，是否就能证明程序对了？」「给这个小实验留下一份能照着做的说明」。

临时环境变量的保存 / 恢复代码和 PowerShell 版本差异可查。重点是只改实验需要的那一处、用完恢复；新进程取得父进程当时环境的副本，后来在另一终端改同名值不会自动同步，A10 将沿实际进程展开。